

UCF Physics: AST 5765/4762: (Advanced) Astronomical Data Analysis
Homework 3: Python Practice, Part 2
Due Wednesday, September 16, 2:30pm.

Goals: Become sufficiently familiar with Python to:

1. Create 1D plots
2. Write and document functions and procedures with good coding style

Handy resources:

1. `np.lookfor("keyword")`: look up functions by word or phrase in docstring
2. remember that `help()` will give you the docstring of any function
3. Chapter1-4 of <https://ssb.stsci.edu/perry/pydatatut.pdf>
4. ThinkPython 2e <https://greenteapress.com/wp/think-python-2e/>

Do not use loops for any problem on this assignment. Remember to open your `0-ast4762-yourname.log` and start logging what you are doing. Also, remember that 'yourusername' should be replaced by the username you have chosen.

1. **(5 points)** Use your terminal to make a new folder `hw3_yourname/` under your `ast4762/homework/` folder. Explain in your log how you did it. For this homework your main homework file is a Python file named `hw3_<username>.py`. If you prefer to work on a notebook you will start with `hw3_<username>.ipynb`, but remember to save it as a `.py` file before submission. Save it in your homework (`hw3/`) folder. Remember to commit your files and push to GitHub as often as needed (also, great backup!).

Your name, assignment number, and the date should appear as comments at the top of the file. At the start of every problem, print the problem number using the appropriate Python command. Any remarks or written answers you may make should be printed out using the appropriate Python command. If you need to comment something in the code (for coding clarity), do so with a normal comment (i.e., `# this is a comment`).

Remember that using comments is good coding practice, and it will be taken into account during the grading process.

2. **(36 points total)** Write a function called `square` that returns the square of its input (which may be a scalar or an array of any dimension or numerical type).

Start by creating your support functions file.

- (a) **(2 points)** Copy the template we provide in `module_template_with_docstring.py` from our `homework/HW3/` folder to create the content of your support functions file. Save your support file following the naming conventions of the class (see `homework_format_rules_25.pdf`, page 2, section 2.1).

- (b) **(3 points)** Study the structure of the template carefully. This is how you want your support file to be structured, ideally. What are the parts that need to be included? Describe them and explain why you think they are there in your log.
- (c) **(5 points)** Make the template suitable for this homework assignment. Replace and remove imports as needed. You will be cleaning up this file more in the following parts. Note that this file will contain multiple functions. Failure to clean the file up properly will result in loss of points.

Now, we will start writing our first function of this class.

- (d) **(2 points)** Name the first function in the file, `square`.
`square` takes as input a scalar or array of any dimension or numerical type and returns its square.
Commit to GitHub. Explain how you did all this in your log.
- (e) **(8 points)** Write the code for your function `square`. Document the function, following the template as well as you can. Be sure to include *at least* one example in your docstring! Copy-paste the answer so it is exactly the same in your docstring as in Python.
- (f) **(8 points)** Test and iterate until your code works. Document in your log any bugs you find and the test cases you used.
Commit to Git/GitHub every time you do a major update to your code. Remember to *test at least one example of all possible input data types!*
- (g) **(2 points)** Update the example and docstring as needed. Commit one final time to GitHub. Make a screenshot of the Git log history showing the updates you did to your file. Name the screenshot appropriately and save it in your homework folder.

Use your function!

- (h) **(3 points)** In your main homework file (`hw3_<username>.py` or `hw3_<username>.ipynb`) create an array `test_square_1` with integers from 0 to 9. Import and call function `square` to square array `test_square_1` and print the result.
- (i) **(3 points)** In a single line, create a 5x5 array `test_square_2` that contains floats from 0 to 25. Call function `square` to square array `test_square_2` and print the result.

3. **(20 points)** In your support function file, write another function called `squareplot`. `Squareplot`'s purpose is to plot the squares of numbers. `squareplot` should have

three positional arguments: the low end of the range it will plot, the high end of the range (inclusive!) it will plot, and the number of points to plot over this range. It should also have one optional argument, `saveplot=False`. The function should then:

- a) Create an array `x` of evenly spaced points from the low up to, and including, the high end of the range.
- b) Call function `square` exactly once to get array `y` that is the square of array `x`.
- c) Plot `y` vs `x`.
- d) If `saveplot` is not `False`, the function should save the plot image as a PDF in the file name given by the user through the `saveplot` argument (note that the filename should not be hardcoded in the function!).

The horizontal and vertical axes of the plot should be labeled “Input” and “Output”, respectively, and the title of the plot should be “Square Function”.

Make sure to document the function (in your function’s docstring) and include an example. Remember to commit your function into GitHub as soon as you have a file, and several times as you write it and debug it. Also, remember to document bugs you find and updates you make in your log.

In the main homework file, call `squareplot` to plot (in the same plot) the squares of the numbers 1, 2.5, 4, 5.5, and 7, and save to an appropriately named file. Remember that you might need to restart your Jupyter notebook kernel to make it re-read your changed files and see your updates.

4. (10 points) Make a rubric for problem 3. Consider the difficulty level of each step, and assign a total of 20 points to the various steps. Then, break each step into sub-steps worth one to a few points. Pose rubric-style questions that would lead the evaluator to award full points only for work that was done well, and partial credit for work that was only partially correct. Write your rubric in a separate `.txt` or `.pdf` file. Include it in your submission. What file formats will the rubric for this question ask for? (Note: There are many reasonable ways to divide the points and ask the questions. But, they would all award full credit for a correct answer and roughly the same amount of partial credit for partially correct answers. All reasonable responses to this question will get full credit. The goal here is to get you to think like an evaluator, so you can guide your own work on future assignments, and also in tasks like writing telescope and funding proposals, which may have rubrics, but don’t publish them.)

5. (10 points) Test that your code works on Stokes. If you worked on a Jupyter notebook, make sure to save it as a `.py` file. Then:

- a) Log in to Stokes. Explain how you did this in your log.
- b) Make a folder `hw3_<yourname>`. Explain how you did this in your log.

- c) Copy your `.py` files to Stokes. Explain how you did this in your log.
- d) Test it! Does it run without issues?
 - a. Yes! Great. Move on to the next steps
 - b. No. Debug!
- e) Don't forget to commit and push to GitHub. Explain what you did to do that in your log.
- f) Make a screenshot that shows your logging history, showing that you committed the file and added it to your `hw3_<yourname>` folder (remember to use an appropriate name for the screenshot! Also, don't forget to copy it to your Stokes folder, too).
- g) If needed, copy the rest of the content of your `homework/hw3_<yourname>` folder to your Stokes folder (e.g., any screenshots, etc., that might have been requested).
- h) Explain how you will do the following in your log and then copy the log into your Stokes folder:
 - a. Copy the log into the Stokes folder.
 - b. Make a `tar.gz` of the folder on Stokes and explain how you did this in your log.
 - c. Download the `tar.gz` file into your local `handin/` folder.
 - d. Submit the file to Webcourses.
- i) Copy your log into your Stokes folder, `tar.gz` your file, and submit your homework.